

1. Create one Jenkins job using the below code and create three stages.

- **stage1:** Git clone to download the source code.
- **stage2:** Sonarqube Integration to check the quality of code.
- **stage3:** Slack Integration to send the alerts to slack.

Source Code: <https://github.com/betawins/VProfile-1.git>

1. Objective

The objective of this task is to create a Jenkins Freestyle job that integrates GitHub, SonarQube, and Slack.

The Jenkins job performs the following activities:

1. Clones the application source code from GitHub.
2. Performs static code analysis using SonarQube.
3. Sends Jenkins build notifications to a Slack channel.
4. Verifies the complete integration by executing a successful Jenkins build.

2. Tools and Technologies Used

- Jenkins
- Git
- GitHub
- SonarQube
- Slack
- AWS EC2
- Jenkins Freestyle Project

3. Application Details

Application: VProfile

GitHub Repository:

<https://github.com/betawins/VProfile-1.git>

Git Branch:

master

Jenkins Job Name:

VProfile-SonarQube-Slack

SonarQube Project Key:

VProfile-1

Slack Channel:

#all-jenkins

4. Implementation Steps

Step 1 – Verify Jenkins

The Jenkins server was verified to ensure that Jenkins was installed, running, and accessible through the Jenkins web interface.

The Jenkins Dashboard was opened and used as the starting point for creating and configuring the Freestyle project.

Step 2 – Install and Verify Required Jenkins Plugins

The required Jenkins plugins were installed and verified from:

Manage Jenkins → Plugins → Installed

The following integrations were required:

- Git integration for retrieving source code from GitHub.
- SonarQube Scanner for performing SonarQube analysis.
- Slack Notification plugin for sending build notifications to Slack.

After installation, the plugins were verified from the Installed Plugins section.

Step 3 – Configure SonarQube Server in Jenkins

SonarQube was configured globally in Jenkins from:

Manage Jenkins → System → SonarQube servers

The SonarQube server configuration was added with the following details:

SonarQube Server URL:

http://172.31.12.251:9000

A Jenkins Secret Text credential containing the SonarQube authentication token was configured and selected for authentication.

The credential ID used was:

sonarqube-token

This configuration allows Jenkins jobs to communicate with the SonarQube server securely.

Step 4 – Configure SonarQube Scanner

The SonarQube Scanner tool was configured from:

Manage Jenkins → Tools

A scanner installation named:

SonarScanner

was configured with automatic installation enabled.

The configured scanner version was:

SonarQube Scanner 8.1.0.6389

This scanner is used by Jenkins to perform the source-code analysis and send the analysis results to the configured SonarQube server.

Step 5 – Create Jenkins Freestyle Project

A new Jenkins job was created from:

Jenkins Dashboard → New Item

The project name was:

VProfile-SonarQube-Slack

The project type selected was:

Freestyle project

The job was created successfully and then opened for configuration.

Step 6 – Configure GitHub Source Code Management

Inside the Jenkins job configuration, the **Source Code Management** section was configured using Git.

The following repository was provided:

`https://github.com/betawins/VProfile-1.git`

No GitHub credentials were required because the repository is publicly accessible.

The branch specification was configured as:

`*/master`

With this configuration, Jenkins retrieves the source code from the VProfile GitHub repository whenever the job is executed.

Step 7 – Configure SonarQube Scanner Build Step

A build step was added using:

Build Steps → Execute SonarQube Scanner

The SonarQube analysis properties were configured as follows:

`sonar.projectKey=VProfile-1`

`sonar.projectName=VProfile-1`

`sonar.sources=.`

Purpose of the properties

sonar.projectKey

Identifies the project uniquely in SonarQube.

sonar.projectName

Defines the project name displayed in the SonarQube dashboard.

sonar.sources

Specifies the source directory to be analyzed. The value `.` indicates the current Jenkins workspace.

The JDK option was kept as:

(Inherit From Job)

The Path to Project Properties field was left blank because the SonarQube analysis properties were provided directly in the Jenkins job configuration.

Step 8 – Configure Slack Integration

Slack integration was configured globally in Jenkins from:

Manage Jenkins → System → Slack

The following configuration was used:

Workspace:

Jenkins

Credential:

Slack Jenkins Bot Token

Default channel / member ID:

all-jenkins

The **Custom slack app bot user** option was enabled because a custom Jenkins Slack application was being used.

The Slack connection was tested using **Test Connection**.

The connection test returned:

Success

This confirmed that Jenkins was successfully connected to the Slack workspace and was able to communicate with the configured channel.

Step 9 – Configure Slack Notifications for the Jenkins Job

The Slack notification configuration was added to the VProfile-SonarQube-Slack job under:

Post-build Actions → Slack Notifications

The following notification options were enabled:

- Notify Build Start
- Notify Success
- Notify Every Failure

These notifications allow the team to receive information about the Jenkins build status directly in Slack.

The remaining notification options were left disabled because they were not required for this implementation.

Step 10 – Save the Jenkins Job Configuration

After configuring GitHub, SonarQube, and Slack, the Jenkins job configuration was reviewed and saved.

The final job therefore contained three main integration components:

GitHub

↓

Jenkins

↓

SonarQube Analysis

↓

Build Result

↓

Slack Notification

Step 11 – Execute the Jenkins Job

The VProfile-SonarQube-Slack job was executed using:

Build Now

Jenkins created a new build and started retrieving the source code from the configured GitHub repository.

The source code was checked out into the Jenkins workspace:

`/var/lib/jenkins/workspace/VProfile-SonarQube-Slack`

The Git checkout completed successfully.

Step 12 – Execute SonarQube Code Analysis

After the source code was checked out, Jenkins executed the configured SonarQube Scanner build step.

The scanner connected to the configured SonarQube server and analyzed the VProfile source code.

The analysis completed successfully and the results were published to the SonarQube project:

VProfile-1

The SonarQube dashboard was then opened to verify that the analysis results were available.

Step 13 – Verify SonarQube Dashboard

The VProfile-1 project was verified in SonarQube.

The dashboard displayed code-quality metrics including:

- Lines of Code
- Security rating
- Reliability rating
- Maintainability rating
- Code coverage
- Code duplication

The presence of these results confirmed that the Jenkins job successfully performed the SonarQube analysis and uploaded the results to the SonarQube server.

Step 14 – Verify Slack Notification

The Slack channel:

#all-jenkins

was opened after executing the Jenkins job.

Slack notifications were successfully received for the Jenkins build.

The notifications confirmed the build status, including the successful completion of the Jenkins job.

This verified the Jenkins-to-Slack integration.

5. Build Verification

The Jenkins build completed successfully with the following final status:

Finished: SUCCESS

The complete workflow was successfully verified:

GitHub Repository

↓

Git Checkout

↓

Jenkins Workspace

↓

SonarQube Analysis

↓

SonarQube Dashboard

↓

Jenkins Build Result

↓

Slack Notification

6. Observations

1. Jenkins successfully cloned the VProfile source code from the public GitHub repository.
2. The source code was stored in the Jenkins job workspace.
3. SonarQube Scanner successfully analyzed the application source code.
4. SonarQube successfully received and displayed the analysis results.
5. Jenkins successfully communicated with the Slack workspace.
6. Slack notifications were successfully delivered to the #all-jenkins channel.
7. The complete Jenkins build finished with a **SUCCESS** status.

7. Final Result

The VProfile-SonarQube-Slack Jenkins Freestyle job was successfully implemented and tested.

The three required integrations were verified successfully:

Integration	Result
GitHub Source Code Checkout	Successful
SonarQube Code Analysis	Successful
Slack Build Notification	Successful
Jenkins Build	Successful

Therefore, the complete **GitHub → Jenkins → SonarQube → Slack** workflow was successfully implemented.

```
[root@ip-172-31-11-114 ~]# sudo systemctl start jenkins
sudo systemctl status jenkins --no-pager
[root@ip-172-31-11-114 ~]# sudo systemctl status jenkins --no-pager
• jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-10-06 06:58:36 UTC; 7s ago
     Main PID: 5798 (java)
       Tasks: 50 (limit: 4543)
      Memory: 289.9M
         CPU: 13.133s
    CGroup: /system.slice/jenkins.service
            └─5798 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpP...

Oct 06 06:58:35 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:35.497+0000 [id=39] INFO jenkins...plugins
Oct 06 06:58:35 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:35.503+0000 [id=36] INFO jenkins...ensions
Oct 06 06:58:35 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:35.920+0000 [id=39] INFO h.p.b.g...not set
Oct 06 06:58:36 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:36.808+0000 [id=39] INFO jenkins... loaded
Oct 06 06:58:36 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:36.809+0000 [id=38] INFO jenkins...adapted
Oct 06 06:58:36 ip-172-31-11-114.ec2.internal jenkins[5798]: 2026-10-06 06:58:36.810+0000 [id=38] INFO jenkins...ll jobs
```

 Jenkins

[+ New Item](#)

[Build History](#)

Build Queue

No builds in the queue.

Build Executor Status

0/2

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

Set up a distributed build

Set up an agent

Configure a cloud

Learn more about distributed builds

REST API

Jenkins 2.580.1

Jenkins

Manage Jenkins

System

Search

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is `http://localhost:9000`

http://172.31.12.251:9000/

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

SonarQube Jenkins Token

+ Add

Advanced

Save

Apply

Jenkins

Manage Jenkins

Tools

Search

SonarQube Scanner installations

SonarQube Scanner installations Edited

+ Add SonarQube Scanner

SonarQube Scanner

Name

SonarScanner

Install automatically

?

Install from Maven Central

Version

SonarQube Scanner 8.1.0.6389

+ Add Installer

+ Add SonarQube Scanner

Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

All

S	W	Name	Last Success	Last Failure	Last Duration
		VProfile-SonarQube-Slack	19 min #2	N/A	15 sec

Icon: S M L

 Jenkins

VPProfile-SonarQube-Slack

Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Git

Repositories

Repository URL

https://github.com/betawins/VPProfile-1.git

Credentials

- none -

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/master

+ Add Branch

 Jenkins

VPProfile-SonarQube-Slack

Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute SonarQube Scanner

JDK

JDK to be used for this SonarQube analysis

(Inherit From Job)

Path to project properties

Analysis properties

sonar.projectKey=VPProfile-1
sonar.projectName=VPProfile-1
sonar.sources=

Additional arguments

JVM Options

 Jenkins

VPProfile-SonarQube-Slack

Configure

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Slack Notifications

☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted

☐ Notify Not Built

☐ Notify Unstable

☐ Notify Regression

☒ Notify Every Failure

☐ Notify Back To Normal

Advanced

+ Add post-build action

Save

Apply

Jenkins / VProfile-SonarQube-Slack #2

Status
Changes
Console Output
Delete build #2
Git Build Data
Edit build information
Previous Build

Console

Download Copy View as plain text

```
Started by user unknown or anonymous
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/VProfile-SonarQube-Slack
The recommended git tool is: NONE
No credentials specified
> /usr/bin/git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/VProfile-SonarQube-Slack/.git # timeout=10
Fetching changes from the remote Git repository
> /usr/bin/git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> /usr/bin/git --version # timeout=10
> git --version # 'git version 2.50.1'
> /usr/bin/git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> /usr/bin/git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffaeef82b81ae721ec883901 (refs/remotes/origin/master)
> /usr/bin/git config core.sparsecheckout # timeout=10
> /usr/bin/git checkout -f afc137167b41aaaffaeef82b81ae721ec883901 # timeout=10
Commit message: "Update README.md"
> /usr/bin/git rev-list --no-walk afc137167b41aaaffaeef82b81ae721ec883901 # timeout=10
[VProfile-SonarQube-Slack] $ /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/bin/sonar-scanner -
Dsonar.host.url=http://172.31.12.251:9000/ -Dsonar.projectKey=VProfile-1 -Dsonar.projectName=VProfile-1 -Dsonar.sources=. -
Dsonar.projectBaseDir=/var/lib/jenkins/workspace/VProfile-SonarQube-Slack
08:18:43.290 INFO Scanner configuration file: /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/conf/sonar-scanner.properties
08:18:43.292 INFO Project root configuration file: NONE
08:18:43.303 INFO SonarScanner CLI 8.1.0.6389
08:18:43.308 INFO Linux 6.18.51-120.163.amzn2023.x86_64 amd64
08:18:44.519 INFO Communicating with SonarQube Community Build 26.9.0.129388
08:18:44.520 INFO JRE provisioning: os[linux], arch[x86_64]
08:18:44.926 INFO Starting SonarScanner Engine...
08:18:44.927 INFO Java 21.0.11 Full-time Adoption (64-bit)
```

← → ↻ ⚠ Not secure 3.236.178.137:8080/job/VProfile-SonarQube-Slack/2/console 🔍 ☆ ⓘ

Jenkins / VProfile-SonarQube-Slack #2

```
08:18:43.462 INFO Sensor: IAC Project Sensor [iac]
08:18:57.409 INFO Sensor: IAC Project Sensor [iac] (done) | time=1ms
08:18:57.410 INFO Sensor: JavaProjectSensor [java]
08:18:57.411 INFO Sensor: JavaProjectSensor [java] (done) | time=1ms
08:18:57.411 INFO Sensor: Zero Coverage Sensor
08:18:57.413 INFO Sensor: Zero Coverage Sensor (done) | time=1ms
08:18:57.419 INFO Sensor: Java CPD Block Indexer
08:18:57.454 INFO Sensor: Java CPD Block Indexer (done) | time=40ms
08:18:57.455 INFO ----- Gather SCA dependencies on project
08:18:57.460 INFO Dependency analysis skipped
08:18:57.476 INFO CPD Executor 8 files had no CPD blocks
08:18:57.477 INFO CPD Executor Calculating CPD for 14 files
08:18:57.497 INFO CPD Executor CPD calculation finished (done) | time=10ms
08:18:57.499 INFO SCM revision ID 'afc137167b41aaaffaeef82b81ae721ec883901'
08:18:57.622 INFO Analysis report generated in 120ms, dir size=680.2 KB
08:18:57.688 INFO Analysis report compressed in 67ms, zip size=152.1 KB
08:18:57.738 INFO Analysis report uploaded in 49ms
08:18:57.740 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://172.31.12.251:9000/dashboard?id=VProfile-1
08:18:57.741 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
08:18:57.741 INFO More about the report processing at http://172.31.12.251:9000/api/ce/task?id=2f64d565-88b6-4703-974b-c89079828562
08:18:57.838 INFO Analysis total time: 11.196 s
08:18:57.842 INFO SonarScanner Engine completed successfully
08:18:57.865 INFO EXECUTION SUCCESS
08:18:57.866 INFO Total time: 14.578s
[Slack Notifications] found #1 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS
```

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality gate

Passed 1

Failed 0

Security

A ≥ 0 info issues 0

B ≥ 1 low issue 0

C ≥ 1 medium issue 1

D ≥ 1 high issue 0

E ≥ 1 blocker issue 0

Reliability

Search projects Perspective Overall Status Sort by Name 1 project(s)

Create Project

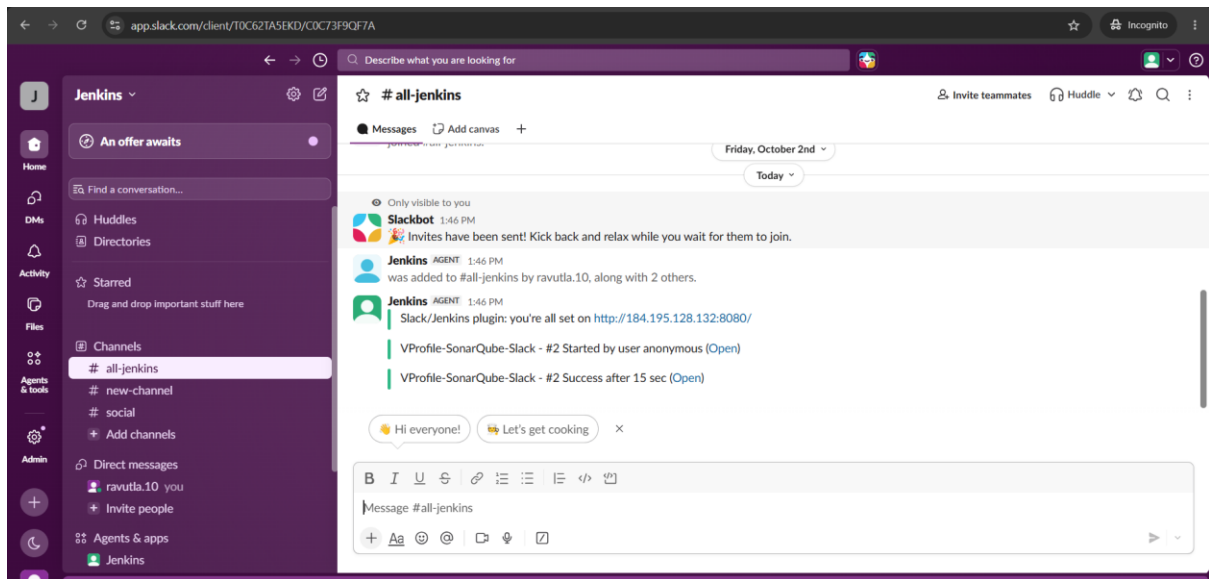
VProfile-1 Public

Last analysis: 21 minutes ago - 1.8k Lines of Code - JSP, Java, ...

9 Security 40 Reliability 51 Maintainability 0.0% Coverage 2.6% Duplications

1 of 1 shown

SonarQube™ technology is powered by SonarSource Sàrl Community Build - v26.9.0.129388 - MQR MODE LGPL v3 Community Documentation Plugins Web API



2. Create one Jenkins job using the below code and create three stages.

- **stage1: Git clone to download the source code.**
- **stage2: Sonarqube Integration to check the quality of code.**
- **stage3: Slack Integration to send the alerts to slack.**

Source Code: <https://github.com/betawins/hiring-app.git>

1. Objective

The objective of this task is to create a Jenkins Freestyle job for the Hiring App application and integrate it with GitHub, SonarQube, and Slack.

The Jenkins job performs the following activities:

1. Retrieves the Hiring App source code from GitHub.
2. Checks out the correct application branch.
3. Performs static code analysis using SonarQube.
4. Sends Jenkins build notifications to Slack.
5. Verifies the complete integration by executing a successful Jenkins build.

2. Tools and Technologies Used

- Jenkins
 - Git
 - GitHub
 - SonarQube
 - Slack
 - AWS EC2
 - Jenkins Freestyle Project
-

3. Application Details

Application: Hiring App

GitHub Repository:

<https://github.com/betawins/hiring-app.git>

Git Branch:

main

Jenkins Job Name:

Hiring-App-SonarQube-Slack

SonarQube Project Key:

hiring-app

SonarQube Project Name:

hiring-app

Slack Channel:

#all-jenkins

4. Implementation Steps

Step 1 – Create Jenkins Freestyle Project

A new Jenkins job was created from:

Jenkins Dashboard → New Item

The project name was entered as:

Hiring-App-SonarQube-Slack

The project type selected was:

Freestyle project

The project was created successfully and the job configuration page was opened.

Step 2 – Configure GitHub Source Code Management

Inside the Hiring-App-SonarQube-Slack job, the **Source Code Management** section was configured using Git.

The following repository URL was provided:

<https://github.com/betawins/hiring-app.git>

No GitHub credentials were required because the repository is publicly accessible.

Initially, the branch was configured as:

`*/master`

However, when the job was executed, Jenkins returned the following error:

ERROR: Couldn't find any revision to build.

The GitHub repository was checked and it was found that the application uses the main branch instead of master.

Therefore, the branch configuration was corrected to:

`*/main`

The final Git configuration was:

Repository URL:

<https://github.com/betawins/hiring-app.git>

Credentials:

- none -

Branch Specifier:

`*/main`

After correcting the branch, Jenkins was able to successfully retrieve the application source code.

Step 3 – Configure SonarQube Scanner Build Step

A SonarQube Scanner build step was added from:

Build Steps → Add build step → Execute SonarQube Scanner

The following analysis properties were configured:

sonar.projectKey=hiring-app

sonar.projectName=hiring-app

sonar.sources=.

Purpose of the properties

sonar.projectKey

Uniquely identifies the Hiring App project in SonarQube.

sonar.projectName

Defines the name of the project displayed in the SonarQube dashboard.

sonar.sources

Specifies the source files to be analyzed. The value . represents the current Jenkins workspace.

The JDK configuration was kept as:

(Inherit From Job)

The Path to Project Properties field was left blank because the SonarQube analysis properties were entered directly into the Jenkins job configuration.

Step 4 – Configure Slack Notifications

Slack notifications were configured under:

Post-build Actions → Slack Notifications

The following notification options were enabled:

- Notify Build Start
- Notify Success

- Notify Every Failure

The following options were left disabled because they were not required for this implementation:

- Notify Aborted
- Notify Not Built
- Notify Unstable
- Notify Regression
- Notify Back To Normal

The Slack notification configuration allows Jenkins to automatically notify the development team about the status of the Hiring App build.

Step 5 – Save the Jenkins Job Configuration

After configuring GitHub, SonarQube, and Slack, the Jenkins job configuration was reviewed and saved.

The final job configuration consisted of:

GitHub Repository

↓

Git Checkout

↓

SonarQube Scanner

↓

Jenkins Build Result

↓

Slack Notification

Step 6 – Execute the Jenkins Job

The Hiring-App-SonarQube-Slack job was executed using:

Build Now

Jenkins started a new build and retrieved the source code from the configured GitHub repository.

The source code was checked out into the Jenkins workspace:

`/var/lib/jenkins/workspace/Hiring-App-SonarQube-Slack`

The main branch was successfully checked out.

Step 7 – Verify Git Checkout

During the initial execution, Jenkins was unable to find the master branch and the build failed.

The error was:

Couldn't find any revision to build.

After checking the GitHub repository, the correct branch was identified as main.

The Jenkins branch configuration was changed from:

`*/master`

to:

`*/main`

The job was then executed again.

The second execution successfully retrieved the source code from the main branch.

This confirmed that the GitHub integration was working correctly.

Step 8 – Execute SonarQube Code Analysis

After successfully checking out the source code, Jenkins executed the configured SonarQube Scanner build step.

The scanner analyzed the Hiring App source code and communicated with the configured SonarQube server.

The analysis completed successfully.

The results were published to the SonarQube project:

`hiring-app`

Step 9 – Verify SonarQube Dashboard

The hiring-app project was opened in the SonarQube dashboard after the Jenkins build completed.

The dashboard was checked to verify that the source code analysis results were available.

The SonarQube dashboard provides code-quality information such as:

- Security
- Reliability
- Maintainability
- Code coverage
- Code duplication
- Lines of code

The presence of the analysis results confirmed that Jenkins successfully integrated with SonarQube and published the Hiring App analysis.

Step 10 – Verify Slack Notification

The Slack channel:

#all-jenkins

was opened after executing the Jenkins job.

Slack notifications were successfully received for the Hiring App Jenkins build.

The notification confirmed the build status and demonstrated that Jenkins was successfully integrated with Slack.

5. Build Verification

The corrected Jenkins job was executed again after changing the branch from master to main.

The final build completed successfully with:

Finished: SUCCESS

The complete workflow was verified as:

GitHub

↓

Hiring App - main branch

↓

Jenkins Git Checkout

↓

SonarQube Analysis

↓

Jenkins Build Result

↓

Slack #all-jenkins

6. Issue Encountered and Resolution

Issue – Jenkins Could Not Find Git Revision

During the first build attempt, Jenkins returned:

ERROR: Couldn't find any revision to build.

Cause

The Jenkins job was configured to build:

*/master

However, the Hiring App GitHub repository uses:

main

as its branch.

Resolution

The branch configuration was changed from:

*/master

to:

*/main

The job was executed again after saving the configuration.

The second build successfully cloned the repository, performed the SonarQube analysis, and sent the Slack notification.

Observation

This demonstrates the importance of verifying the actual branch name in the Git repository before configuring Jenkins SCM.

7. Observations

1. Jenkins successfully connected to the public Hiring App GitHub repository.
 2. The correct Git branch was identified as main.
 3. The initial build failed because Jenkins was configured to use the incorrect master branch.
 4. After changing the branch to main, Git checkout completed successfully.
 5. SonarQube successfully analyzed the Hiring App source code.
 6. SonarQube successfully displayed the project analysis results.
 7. Jenkins successfully sent build notifications to the #all-jenkins Slack channel.
 8. The final Jenkins build completed successfully.
-

8. Final Result

The Hiring-App-SonarQube-Slack Jenkins Freestyle job was successfully implemented and tested.

The three required integrations were successfully verified:

Integration	Result
GitHub Source Code Checkout	Successful
Correct main Branch Checkout	Successful
SonarQube Code Analysis	Successful
Slack Build Notification	Successful
Jenkins Build	Successful

Therefore, the complete **GitHub → Jenkins → SonarQube → Slack** workflow for the Hiring App application was successfully implemented.

Jenkins

All / New Item

Q

New Item

Enter an item name

Hiring-App-SonarQube-Slack

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder

OK

Jenkins

Hiring-App-SonarQube-SL... / Configure

Q

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Git

Repositories

Repository URL

https://github.com/betawins/hiring-app-git

Credentials

- none -

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

+ Add Branch

Save

Apply

← → ↻ Not secure 3.236.178.137:8080/job/Hiring-App-SonarQube-Slack/configure 🔍 ☆ ⓘ ⋮

Jenkins / Hiring-App-SonarQube-SL... / Configure 🔍 ⚙️

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps**
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⋮ Execute SonarQube Scanner

JDK ?
JDK to be used for this SonarQube analysis
(Inherit From Job) ▾

Path to project properties ?

Analysis properties ?

```
sonar.projectKey=hiring-app  
sonar.projectName=hiring-app  
sonar.sources=
```

Additional arguments ?
 ▾

JVM Options ?
 ▾

Save Apply

← → ↻ Not secure 3.236.178.137:8080/job/Hiring-App-SonarQube-Slack/configure 🔍 ☆ ⓘ ⋮

Jenkins / Hiring-App-SonarQube-SL... / Configure 🔍 ⚙️

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions**

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

⋮ Slack Notifications

☒ Notify Build Start

☒ Notify Success

☐ Notify Aborted

☐ Notify Not Built

☐ Notify Unstable

☐ Notify Regression

☒ Notify Every Failure

☐ Notify Back To Normal

Advanced ▾

+ Add post-build action

Save Apply

← → ↻ Not secure 3.236.178.137:8080/job/Hiring-App-SonarQube-Slack/ 🔍 ☆ ⓘ ⋮

Jenkins / Hiring-App-SonarQube-SL... 🔍 ⚙️

Status

</> Changes

Workspace

▶ Build Now

🗑 Delete Project

⚙ Configure

✎ Rename

🔗 SonarQube

Hiring-App-SonarQube-Slack

 SonarQube

Permalinks

Add description

Builds > ...

Today

❌ #1 09:12 ▾

Jenkins

Hiring-App-SonarQube-SL...

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

SonarQube

Builds >

Filter

Today

✓ #2 09:14

✗ #1 09:12

✓

Hiring-App-SonarQube-Slack

Add description

SonarQube

SonarQube Quality Gate

hiring-app Passed

server-side processing: Success

Permalinks

- Last build (#2), 1 min 0 sec ago
- Last stable build (#2), 1 min 0 sec ago
- Last successful build (#2), 1 min 0 sec ago
- Last failed build (#1), 2 min 58 sec ago
- Last unsuccessful build (#1), 2 min 58 sec ago
- Last completed build (#2), 1 min 0 sec ago

Jenkins

Hiring-App-SonarQube-SL... / #2

Status

Changes

Console Output

Delete build #2

Git Build Data

Edit build information

Previous Build

Download

Copy

View as plain text

Console

Started by user unknown or anonymous
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/Hiring-App-SonarQube-Slack
The recommended git tool is: NONE
No credentials specified
> /usr/bin/git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Hiring-App-SonarQube-Slack/.git # timeout=10
Fetching changes from the remote Git repository
> /usr/bin/git config remote.origin.url https://github.com/betawins/hiring-app.git # timeout=10
Fetching upstream changes from https://github.com/betawins/hiring-app.git
> /usr/bin/git --version # timeout=10
> git --version # 'git version 2.50.1'
> /usr/bin/git fetch --tags --force --progress -- https://github.com/betawins/hiring-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> /usr/bin/git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9405b107dffdf68e69fe953ealf9669784f9d6a (refs/remotes/origin/main)
> /usr/bin/git config core.sparsecheckout # timeout=10
> /usr/bin/git checkout -f 9405b107dffdf68e69fe953ealf9669784f9d6a # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Hiring-App-SonarQube-Slack] \$ /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/bin/sonar-scanner -
Dsonar.host.url=http://172.31.12.251:9000/ ***** -Dsonar.projectKey=hiring-app -Dsonar.projectName=hiring-app -Dsonar.sources=. -
Dsonar.projectBaseDir=/var/lib/jenkins/workspace/Hiring-App-SonarQube-Slack
09:14:07.103 INFO Scanner configuration file: /var/lib/jenkins/tools/hudson.plugins.sonar.SonarRunnerInstallation/SonarScanner/conf/sonar-scanner.properties
09:14:07.105 INFO Project root configuration file: NONE
09:14:07.116 INFO SonarScanner CLI 8.1.0.6389
09:14:07.122 INFO Linux 6.18.51-120.163.amzn2023.x86_64 amd64
09:14:08.360 INFO Communicating with SonarQube Community Build 26.9.0.129388
09:14:08.361 INFO JRE provisioning: os[linux], arch[x86_64]
09:14:08.774 INFO Starting SonarScanner Engine...
09:14:08.775 INFO Java 21.0.11 Eclipse Adoptium (64-bit)

Jenkins

Hiring-App-SonarQube-SL... / #2

Not secure

3.236.178.137:8080/job/Hiring-App-SonarQube-Slack/2/console

09:14:13.890 INFO Sensor JavaProjectSensor [java]
09:14:13.891 INFO Sensor JavaProjectSensor [java] (done) | time=1ms
09:14:13.891 INFO Sensor Zero Coverage Sensor
09:14:13.891 INFO Sensor Zero Coverage Sensor (done) | time=0ms
09:14:13.891 INFO ----- Gather SCA dependencies on project
09:14:13.907 INFO Dependency analysis skipped
09:14:13.909 SCM Publisher SCM provider for this project is: git
09:14:13.911 SCM Publisher 4 source files have been analyzed (done) | time=302ms
09:14:14.216 INFO CPD Publisher 4/4 source files have been analyzed (done) | time=302ms
09:14:14.216 INFO CPD Executor 1 file had no CPD blocks
09:14:14.216 INFO CPD Executor Calculating CPD for 0 files
09:14:14.217 INFO CPD Executor CPD calculation finished (done) | time=0ms
09:14:14.222 INFO SCM revision ID '9405b107dffdf68e69fe953ealf9669784f9d6a'
09:14:14.440 INFO Analysis report generated in 213ms, dir size=295.2 kB
09:14:14.459 INFO Analysis report compressed in 20ms, zip size=35.4 kB
09:14:14.655 INFO Analysis report uploaded in 196ms
09:14:14.658 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://172.31.12.251:9000/dashboard?id=hiring-app
09:14:14.659 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
09:14:14.661 INFO More about the report processing at http://172.31.12.251:9000/api/ce/task?id=1e01b49-a49a-4591-b0cb-748c5caaf38c
09:14:14.669 INFO Analysis total time: 4.195 s
09:14:14.673 INFO SonarScanner Engine completed successfully
09:14:14.692 INFO EXECUTION SUCCESS
09:14:14.693 INFO Total time: 7.592s
[Slack Notifications] found #1 as previous completed, non-aborted build
[Slack Notifications] will send OnSuccessNotification because build matches and user preferences allow it
Finished: SUCCESS

⚠️ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#)

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. [Learn more in SonarQube documentation](#)



ProjectsIssuesRulesQuality profilesQuality gatesAdministrationMore

My FavoritesAll

Filters

Quality gate

Passed2

Failed0

Security

A ≥ 0 info issues0

B ≥ 1 low issue1

C ≥ 1 medium issue1

D ≥ 1 high issue0

E ≥ 1 blocker issue0

Reliability

Search projects

PerspectiveOverall Status

Sort byName

Create Project

2 project(s)

hiring-appPublicPassed

Last analysis: 1 minute ago · 45 Lines of Code · XML, Docker, ...

B2

C3

A0

A—

—

0.0%

SecurityReliabilityMaintainabilityHotspots ReviewedCoverageDuplications

VProfile-1PublicPassed

Last analysis: 56 minutes ago · 1.8k Lines of Code · JSP, Java, ...

C9

E40

A51

A—

0.0%

2.6%

SecurityReliabilityMaintainabilityHotspots ReviewedCoverageDuplications

app.slack.com/client/TOC62TA5EKD/COC73F9QF7A

Describe what you are looking for

Incognito

Jenkins

An offer awaits

Find a conversation...

Huddles

Directories

Starred

Drag and drop important stuff here

Channels

all-jenkins

new-channel

social

Add channels

Direct messages

ravutla.10 you

Invite people

Agents & apps

Jenkins

all-jenkins

Invite teammates

Huddle

Messages

Add canvas

Today

Jenkins AGENT 1:46 PM

was added to #all-jenkins by ravutla.10, along with 2 others.

Jenkins AGENT 1:46 PM

Slack/Jenkins plugin: you're all set on <http://184.195.128.132:8080/>

VProfile-SonarQube-Slack - #2 Started by user anonymous (Open)

VProfile-SonarQube-Slack - #2 Success after 15 sec (Open)

Jenkins AGENT 2:42 PM

Hiring-App-SonarQube-Slack - #1 Failure after 0.97 sec (Open)

Hiring-App-SonarQube-Slack - #2 Started by user anonymous (Open)

Hiring-App-SonarQube-Slack - #2 Success after 8.7 sec (Open)

Hi everyone!

Let's get cooking

Message #all-jenkins